

Whack-A-Mole Technical Documentation

1. OOPs Design and Justification

Inheritance Hierarchy

The core design centers on the abstract class `HoleOccupant`, which serves as the base for all objects that can appear from holes in the game grid.

Why an Abstract Class (not an Interface)?

An abstract class was chosen over an interface because all occupants share common state and behavior:

- Shared State:** `visible` (boolean), `timeRemaining` (int), `DEFAULT_LIFETIME` (constant)
- Shared Behavior:** `hide()`, `tick()`, `isVisible()`, `getTimeRemaining()`

An interface cannot contain instance fields or concrete method implementations (prior to Java 8 default methods), making it unsuitable for this shared functionality.

Abstract Contract:

```
public abstract int whack();           // Returns score change
public abstract ImageIcon getImage(); // Returns visual representation
public abstract String getTypeName(); // Returns type identifier
```

Concrete Subclasses

Class	Score	Lifetime	Special Feature
<code>Mole</code>	+100	3 ticks	Standard target
<code>Bomb</code>	-500	4 ticks	Penalty object
<code>BonusMole</code>	+500	2 ticks	+5 seconds bonus time

Polymorphism in Action

The `GameEngine` maintains a polymorphic collection:

```
private final HoleOccupant[] holes;
```

When processing a click, **no type checking is performed for scoring**:

```
// POLYMORPHIC CALL - behavior determined at runtime
int scoreChange = occupant.whack();
score += scoreChange;
```

The specific `whack()` implementation executes based on the runtime type of the object, demonstrating true polymorphism.

2. Concurrency Strategy

Thread Architecture

The application uses two threads:

- Event Dispatch Thread (EDT):** Handles all GUI operations and user input
- Game Thread:** Runs the `GameEngine` game loop independently

GameEngine as Runnable

```

public class GameEngine implements Runnable {
    @Override
    public void run() {
        while (gameIsRunning && timeRemaining > 0) {
            spawnOccupants();
            tickAllOccupants();
            timeRemaining--;
            Thread.sleep(TICK_INTERVAL); // 1000ms
        }
    }
}

```

Thread Creation

```

gameEngine = new GameEngine(this);
gameThread = new Thread(gameEngine);
gameThread.start();

```

Thread-Safe UI Updates

Critical Rule: The GameEngine thread CANNOT directly modify Swing components.

All UI updates use `SwingUtilities.invokeLater()`:

```

private void updateScoreOnUI(int score) {
    SwingUtilities.invokeLater(() -> callback.onScoreUpdate(score));
}

```

This marshals the update back to the EDT, ensuring thread safety.

Callback Interface Pattern

The `GameCallback` interface enables decoupled communication:

```

public interface GameCallback {
    void onScoreUpdate(int score);
    void onTimeUpdate(int time);
    void onHoleUpdate(int index, HoleOccupant occupant);
    void onGameOver(int finalScore, int highScore);
    void onHighScoreUpdate(int highScore);
}

```

The GUI implements this interface, and the GameEngine calls these methods (via `invokeLater`) to update the display.

Graceful Shutdown

When the window closes:

1. `exitGame()` calls `gameThread.interrupt()`
2. `Thread.sleep()` throws `InterruptedException`
3. Catch block sets `gameIsRunning = false`
4. Thread exits cleanly

```

} catch (InterruptedException e) {
    gameIsRunning = false;
    Thread.currentThread().interrupt();
    return;
}

```

3. Custom Exceptions

HighScoreException (Checked)

Extends: Exception

Purpose: Wraps file I/O errors during score serialization/deserialization.

Rationale: This is a **checked exception** because:

- File I/O errors are **recoverable** - the game can continue with an empty high score list
- The caller (GUI) should be **forced** to handle this scenario
- Provides meaningful error messages to users

Usage:

```
public List<PlayerScore> loadScores() throws HighScoreException {
    try {
        // deserialization code
    } catch (IOException e) {
        throw new HighScoreException("Failed to load scores", e);
    } catch (ClassNotFoundException e) {
        throw new HighScoreException("Invalid score data", e);
    }
}
```

InvalidGameStateException (Unchecked)

Extends: RuntimeException

Purpose: Indicates programmer errors in game logic.

Rationale: This is an **unchecked exception** because:

- Represents **unrecoverable** programming bugs, not runtime conditions
- Should **never** occur in correct code
- Callers shouldn't be forced to catch programmer errors

Usage:

```
if (index < 0 || index >= GRID_SIZE) {
    throw new InvalidGameStateException(
        "Invalid hole index: " + index);
}
```

4. Data Persistence

Serialization Approach

The `PlayerScore` class implements `Serializable`:

```
public class PlayerScore implements Serializable {
    private static final long serialVersionUID = 1L;
    private String playerName;
    private int score;
    private long timestamp;
}
```

Save/Load Operations

Saving:

```
try (ObjectOutputStream oos = new ObjectOutputStream(
    new FileOutputStream("scores.dat"))) {
    oos.writeObject(scores);
}
```

Loading:

```
try (ObjectInputStream ois = new ObjectInputStream(
    new FileInputStream("scores.dat"))) {
    return (ArrayList<PlayerScore>) ois.readObject();
}
```

The entire `ArrayList<PlayerScore>` is serialized as a single object, maintaining the collection structure.

5. Key Design Decisions Summary

Decision	Rationale
Abstract class over interface	Shared state and concrete methods needed
Callback interface pattern	Decouples engine from GUI, enables testing
Volatile <code>gameIsRunning</code> flag	Ensures visibility across threads
Synchronized <code>whackHole()</code> method	Prevents race conditions on score updates
Two custom exception types	Distinguishes recoverable vs unrecoverable errors
ObjectOutputStream/InputStream	Required per specification; handles complex objects